

Milestone H

Acceptance Testing - Test Cases, Results & Issues

Embedded Systems - Face Detection Game Project

SDU - Software Engineering

May 2026

1. Overview

This document covers the acceptance testing phase for the Face Detection Game. The game runs on a Raspberry Pi with a webcam, uses OpenCV-based face detection, tracks a score and strike system, and provides real-time HUD feedback. Physical hardware buttons (Button 0 and Button 1) are used for all in-game controls. Acceptance tests verify that the delivered system meets the requirements defined in earlier milestones.

Testing Targets

- Face detection accuracy and response time
- Game mechanics: scoring, strikes, difficulty scaling
- Game state machine: IDLE, RUNNING, PAUSED, END
- User controls: hardware button 0 and button 1 handling
- HUD display correctness and real-time updates
- Performance: frame rate and input latency
- Hardware integration: LED feedback and GPIO buttons

2. Acceptance Test Cases

A. Face Detection

These tests verify the core detection pipeline. Tests were run against synthetic images (blank frames, random noise) and live camera sessions.

ID	Test Case	Expected Result	Actual Result	Status
A1	Blank frame returns no faces	detect_faces() returns empty list on a 640x480 black image	Empty list returned	PASS
A2	Detection pipeline returns correct types	process_frame() returns (ndarray, int, str)	Types confirmed correct	PASS
A3	No-face message is correct	Message equals 'No faces detected' when 0 faces found	Message matches	PASS
A4	Detector produces consistent results	Same blank image run 3 times yields identical face count	All 3 runs: 0 faces	PASS
A5	Detection works on multiple frame sizes	320x240, 640x480, 1280x720 all process without error	All sizes processed	PASS
A6	Frontal face detected reliably	Face positioned straight toward camera is detected	Detected consistently	PASS
A7	Angled face detection (15-45 degrees)	Face turned 15-45 degrees to the side is detected	Often missed at >20 deg	FAIL

B. Game Mechanics

These tests verify scoring, strikes, difficulty, and game-over logic using the CameraFaceDetector class with hardware disabled.

ID	Test Case	Expected Result	Actual Result	Status
B1	Game starts with zeroed state	After reset_game(): score=0, strikes=0, game_over=False	All values confirmed 0/False	PASS
B2	Score increments correctly	Calling add_score() twice results in score=2	score=2 after 2 calls	PASS
B3	Strike increments correctly	Calling add_strike() twice results in strikes=2	strikes=2 after 2 calls	PASS
B4	Game over at strike 3	game_over becomes True when add_strike() called 3 times	game_over=True at strike 3	PASS
B5	Timer decreases with score	get_switch_time() returns 3.0 at score=0, 2.9 at score=1	Decrease of 0.1s per point confirmed	PASS
B6	Minimum timer floor is 0.5s	After 50+ score increments, get_switch_time() returns 0.5	Floor held at 0.5s	PASS
B7	Timer resets correctly	Elapsed time near 0 immediately after reset_switch_timer()	Elapsed < 0.1s post-reset	PASS

C. Game State and Controls

These tests verify state transitions and hardware button input handling during live gameplay sessions on the Raspberry Pi. Button 0 is used to start the game; Button 1 is used to pause and resume.

ID	Test Case	Expected Result	Actual Result	Status
C1	IDLE state on launch	Camera feed is live, HUD shows waiting prompt	IDLE state displayed correctly	PASS
C2	Button 0 starts the game	Pressing Button 0 transitions state from IDLE to RUNNING	RUNNING state confirmed	PASS
C3	Button 1 pauses and resumes	First press of Button 1 pauses; second press resumes	Toggle works on single press	PASS
C4	Game ends on 3 strikes	END state shows final score and game over message	END state displayed correctly	PASS
C5	Face target adjustable via buttons	Face target adjustable between 1 and 5	Target adjusts within bounds	PASS
C6	Button 0 restarts from END state	Pressing Button 0 in END state resets game to IDLE/RUNNING	Restart works correctly	PASS

D. Performance

Performance tests were conducted on the Raspberry Pi with the connected webcam.

ID	Test Case	Expected Result	Actual Result	Status
D1	Detection latency under 100ms	process_frame() completes within 100ms on 640x480 frame	Avg 35ms per frame	PASS
D2	Frame rate reaches 30 FPS	Live camera loop sustains 30+ FPS	0.5-0.8 FPS observed on Pi	FAIL
D3	No memory leak over 5 minutes	Memory usage stable during continuous 5-minute session	Memory stable	PASS
D4	Button input processed within 50ms	Button press reflected in game state within 50ms	70-120ms observed	FAIL

E. Hardware Integration

Hardware tests verify the GPIO button and LED interface on the Raspberry Pi.

ID	Test Case	Expected Result	Actual Result	Status
E1	HardwareController initializes in simulation	No exception raised when use_hardware=False	Initialized cleanly	PASS
E2	LED feedback on game events	LED changes color on score/strike events	No visible LED color change observed	FAIL
E3	Button event triggers callback	Registered callback invoked when button is pressed	Callback invoked correctly	PASS

3. Test Results Summary

Total tests: 24

Passed: 20

Failed: 4

Pass rate: 83.3%

Core functionality passes. The game detects faces, tracks score and strikes, manages state transitions, and responds to hardware button input. The main issues are the very low frame rate on the Raspberry Pi, slow button input response, limited face angle tolerance, and absent LED color feedback.

4. Identified Issues and Improvements

Issues Found

HIGH**Very Low Frame Rate (D2) - 0.5 to 0.8 FPS on Raspberry Pi**

The detection loop runs full face detection on every frame on the Pi, resulting in only 0.5-0.8 FPS. This makes the game nearly unplayable as the video feed is extremely choppy. Running detection on a downscaled copy, skipping detection on alternate frames, or offloading detection to a background thread would significantly improve throughput.

HIGH**LED Feedback Not Visible (E2) - No color change observed**

The LED is expected to change color on score and strike events, but no visible change was observed during testing. This may be a wiring issue, a GPIO pin misconfiguration, or the LED driver code not being reached during gameplay. The LED circuit and `set_led()` call site should be verified.

HIGH**Button Input Latency (D4) - 70 to 120ms response time**

Hardware button presses are reflected in the game state after 70-120ms instead of the target 50ms. The main loop processes button events only when it polls between frames. Because frames take over a second to process at 0.5-0.8 FPS, button response is effectively frame-rate limited. Moving button polling to a dedicated interrupt thread would decouple it from frame processing.

MEDIUM**Face Angle Tolerance (A7) - Detection fails beyond ~20 degrees**

The Haar Cascade is trained primarily on frontal faces. At angles greater than 20 degrees detection becomes unreliable. Adding a profile-face cascade or switching to a DNN model would improve angle tolerance.

Suggested Improvements

- Run face detection on a half-resolution copy to reduce per-frame cost on the Pi
- Move button polling to a GPIO interrupt callback so it is not blocked by frame processing
- Verify LED wiring and test `set_led()` in isolation to diagnose the missing color feedback
- Add a profile-face Haar Cascade as a second pass for angled faces
- Show a visual countdown (3-2-1) on-screen before each capture phase
- Add a high-score display that persists between sessions

5. Conclusion

The Face Detection Game passes 20 of 24 acceptance tests. All core game logic requirements are met: face detection works, scoring and strikes are correct, state transitions are reliable, and hardware button input is functional.

The three high-priority issues all stem from running heavy OpenCV processing on the Raspberry Pi without optimization. The frame rate of 0.5-0.8 FPS is the most critical problem and directly causes the button latency issue as well. Resolving the frame rate bottleneck should be the first priority. The missing LED feedback should be diagnosed separately as it may be a simple hardware or wiring fix.

Report Date: May 07, 2026

Project: Embedded Systems - Face Detection Game

Milestone: H - Acceptance Testing

Environment: Raspberry Pi, Python 3.8+, OpenCV 4.x